

Best Teacher Platform – System Components Specification

Overview

Best Teacher is a comprehensive web-based platform that connects students (Grades 6–13) with qualified teachers for private tutoring. The system provides a complete learning ecosystem, including user management, content delivery, assessments, lesson booking, payments, and support features. It consists of six integrated full-stack components, each developed by a team member.

Component 2: Content Management

Primary Entities: Course, Lecture, Material (File/Video)

CRUD Operations

Operation	Description
Create	Teacher creates new content: uploads lecture notes, videos, presentations; organizes by subject, grade, and topic.
Read	Students view and download materials (filtered by subject/grade). Teachers view their own uploaded content.
Update	Teacher edits content metadata (title, description) or replaces uploaded files.
Delete	Teacher removes content (soft delete or permanent).

Functional Requirements

- Upload files (PDF, DOC, PPT, MP4, etc.) to cloud storage
- Organize content with metadata: subject, grade, topic, description
- Search and filter content for students
- Display content list with thumbnails/file icons
- Track upload date and last modified

Non-Functional Requirements

- Supported file types and size limits (e.g., max 100MB)
- Secure access: only authenticated users can view
- Fast content delivery via CDN

Integration Points

- Content is linked to teacher (from Auth)
 - May be associated with courses or subjects; could be used by **Quiz** for reference materials.
-

Component 3: Quiz & Assessment Management

CRUD Operations

Operation	Description
Create	Teacher creates a quiz: title, description, deadline, list of questions (MCQ, short answer, true/false), correct answers, and points.
Read	Students view available quizzes (with deadlines). Teachers view their created quizzes and student submissions.
Update	Teacher edits quiz details before publishing; can adjust grades manually if needed.
Delete	Teacher deletes a quiz (with confirmation).

Functional Requirements

- Quiz creation form with dynamic question addition
- Support for multiple question types
- Set deadline and publish/unpublish
- Students take quiz (timer optional)
- Auto-grading for objective questions
- Grade display to student after submission
- Notifications for upcoming deadlines (email/in-app)

Non-Functional Requirements

- Responses saved securely; prevent multiple submissions
- Deadline enforcement
- Scalable for many concurrent quiz attempts

Integration Points

- Quizzes belong to a teacher (from Auth)
- Student submissions link to student profile
- Grades feed into student progress (optional)
- May link to **Content** for related materials

Component 4: Booking / Request System

Primary Entities: Booking

CRUD Operations

Operation	Description
Create	Student creates a booking request by selecting a teacher and an available time slot from the teacher's schedule. Frontend checks slot availability; backend creates a pending booking.
Read	Both student and teacher can view bookings filtered by status (pending, confirmed, completed, cancelled).
Update	Teacher accepts (→ confirmed) or rejects (→ cancelled) a pending request. Student can cancel a pending booking. After lesson, either party marks as completed.
Delete	Not physically deleted; status set to cancelled (soft delete).

Functional Requirements

- Booking request form showing teacher's available slots
- Real-time availability check (UI and backend)
- Accept/reject buttons on teacher dashboard
- Booking lists with status filters
- Mark completed (after scheduled time)
- Optional cancellation reason

Non-Functional Requirements

- **Double-booking prevention:** Atomic database operations ensure no overlapping bookings for the same teacher.
- Only authenticated users can access; only involved parties can update.
- Status transitions follow a strict state machine (pending → confirmed/completed/cancelled).
- Audit trail: timestamps for creation and all status changes.

Integration Points

- Consumes teacher availability from **Auth** (teacher profile).
 - On booking confirmation, triggers payment in **Financial Management**.
 - Completed bookings are passed to **Reviews** for feedback.
-

Component 6: Financial Management

Primary Entities: Transaction, Payment, Payout, Invoice

CRUD Operations

Operation	Description
Create	When a booking is confirmed, a payment record is created. Student completes payment via gateway; transaction stored with status pending → completed. Invoice generated.
Read	Students view their payment history; teachers view earnings and pending payouts; admin views all transactions.
Update	Payment status updated (completed, failed, refunded). Teacher updates payout details (bank account, PayPal). Admin adjusts platform fee configuration.
Delete	Rare; admin may reverse a transaction in case of error (with audit log).

Functional Requirements

- Integration with payment gateway (Stripe/PayPal) for secure payment processing
- Platform fee deduction (percentage or fixed) per transaction
- Teacher payout management: teachers can request withdrawal; admin processes payouts
- Automatic invoice generation (PDF) after successful payment
- Transaction history with filters (date, status, user)
- Refund handling for cancellations (partial/full)

Non-Functional Requirements

- PCI compliance via payment gateway (no raw card data stored)
- Idempotency keys to prevent duplicate charges
- Atomic updates: payment status and booking status must be consistent
- Audit trail for all financial actions
- Encryption of sensitive payout details

Integration Points

- Triggered by **Booking** confirmation
- Provides payment status back to **Booking** (e.g., paid flag)
- Teacher earnings balance stored in **Auth** (teacher profile)
- Invoices link to booking and user